

UNIVERSIDAD AMERICANA



Programación Orientada a Objetos 2

Guía Práctica: Pruebas Unitarias en Aplicaciones Android con Jetpack Compose

Creado por:

Kenneth Alexander Acuña Estrada

Docente:

José Durán García

Managua, 09 de Junio de 2026

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación móvil de Calculadora Pro con una estética Neon-Retro, construida íntegramente con Jetpack Compose para el sistema operativo Android. El propósito fundamental de esta práctica es aplicar los principios de calidad de software mediante la implementación de una Suite de Pruebas Automatizadas.

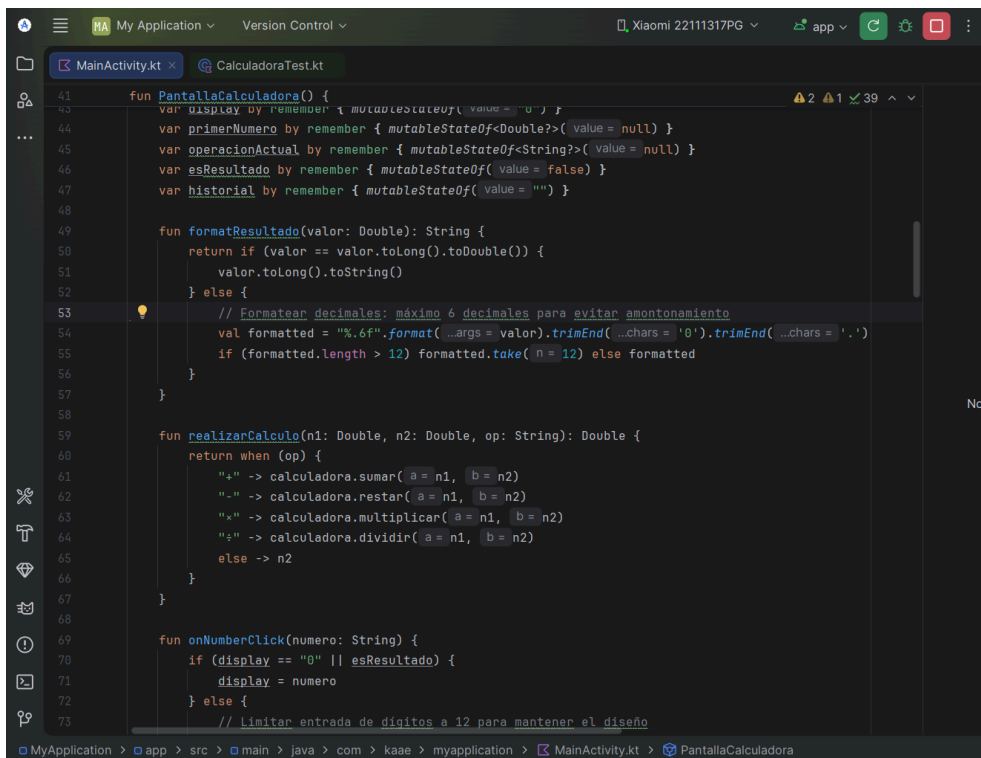
Para garantizar la fiabilidad de la aplicación, el desarrollo se dividió en tres pilares:

- 1. Lógica de Negocio: Una clase robusta encargada de procesar operaciones aritméticas (suma, resta, multiplicación, división y porcentaje).*
- 2. Interfaz de Usuario (UI): Un diseño moderno y minimalista que utiliza estados dinámicos para reflejar el historial y los resultados en tiempo real.*
- 3. Verificación y Validación: El uso de Pruebas Unitarias (JUnit) para asegurar la precisión matemática y Pruebas de Interfaz (Compose Test Rule) para validar la interactividad y el flujo del usuario.*

Este enfoque asegura que la aplicación no solo sea visualmente atractiva, sino técnicamente sólida y libre de errores críticos en sus funciones básicas.

2. Evidencia

1. Aplicación en Ejecución



```
41 fun PantallaCalculadora() {
42     var display by remember { mutableStateOf<String?>("0") }
43     var primerNumero by remember { mutableStateOf<Double?>("0") }
44     var operacionActual by remember { mutableStateOf<String?>("0") }
45     var esResultado by remember { mutableStateOf<Boolean?>("0") }
46     var historial by remember { mutableStateOf<String?>("0") }
47
48     fun formatResultado(valor: Double): String {
49         return if (valor == valor.toLong().toDouble()) {
50             valor.toLong().toString()
51         } else {
52             // Formatear decimales: máximo 6 decimales para evitar amontonamiento
53             val formatted = "%.6f".format(valor).trimEnd( ...chars = '0').trimEnd( ...chars = '.')
54             if (formatted.length > 12) formatted.take( n = 12) else formatted
55         }
56     }
57
58     fun realizarCalculo(n1: Double, n2: Double, op: String): Double {
59         return when (op) {
60             "+" -> calculadora.sumar( a = n1, b = n2)
61             "-" -> calculadora.restar( a = n1, b = n2)
62             "*" -> calculadora.multiplicar( a = n1, b = n2)
63             "/" -> calculadora.dividir( a = n1, b = n2)
64             else -> n2
65         }
66     }
67
68     fun onNumberClick(numero: String) {
69         if (display == "0" || esResultado) {
70             display = numero
71         } else {
72             // Limitar entrada de dígitos a 12 para mantener el diseño
73         }
74     }
75 }
```

10:29 ☑ ...

📶 8%

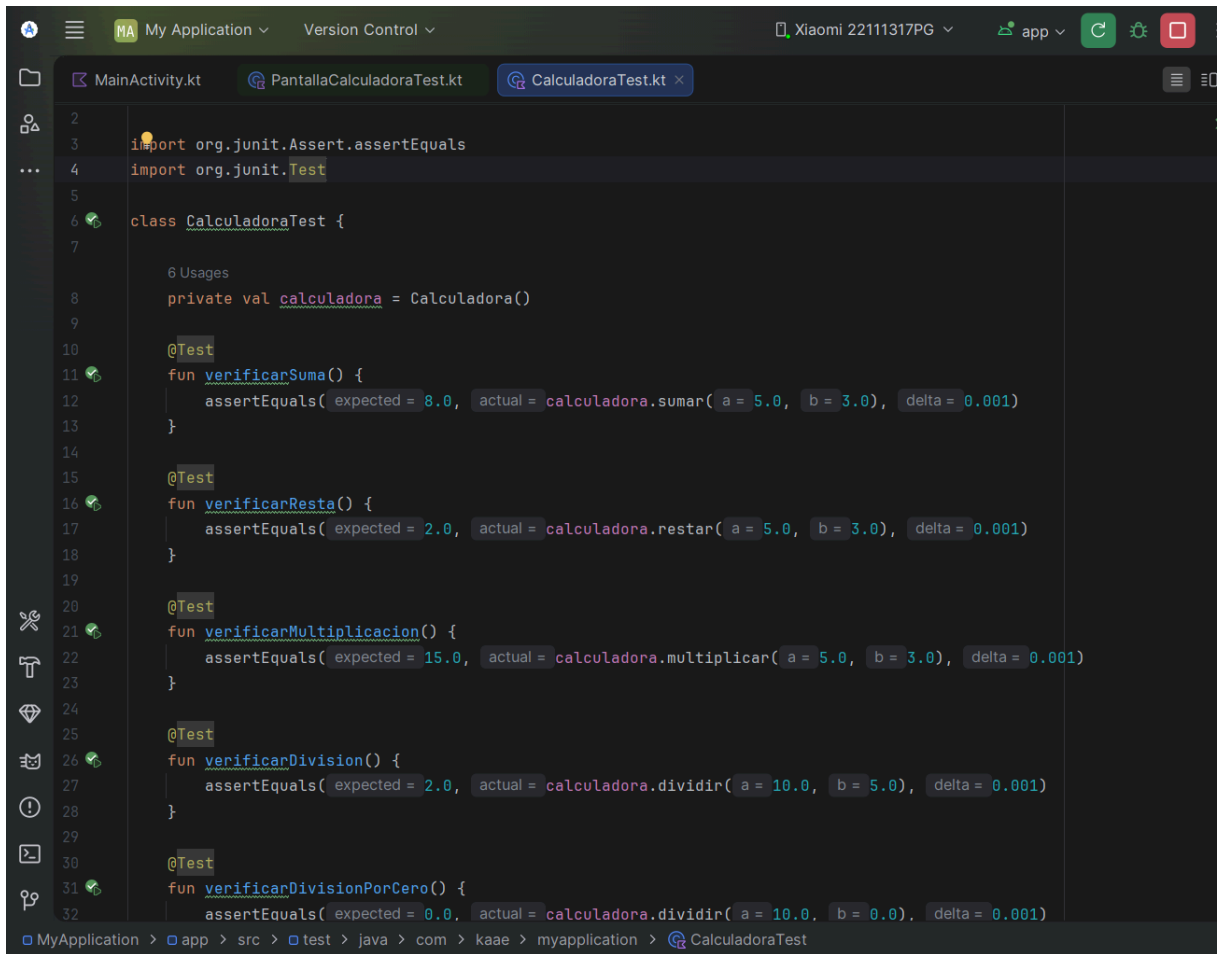
NEON CALC



0

AC	⌫	%	÷
7	8	9	×
4	5	6	-
1	2	3	+
↕	0	.	=

2. Prueba Unitaria



The screenshot shows an IDE with the file `CalculadoraTest.kt` open. The code defines a test class `CalculadoraTest` with five test methods: `verificarSuma`, `verificarResta`, `verificarMultiplicacion`, `verificarDivision`, and `verificarDivisionPorCero`. Each method uses `assertEquals` to verify the output of the calculator's `sumar`, `restar`, `multiplicar`, and `dividir` methods against expected values. The IDE interface includes a top bar with project information, a left sidebar with navigation icons, and a bottom status bar showing the current file path.

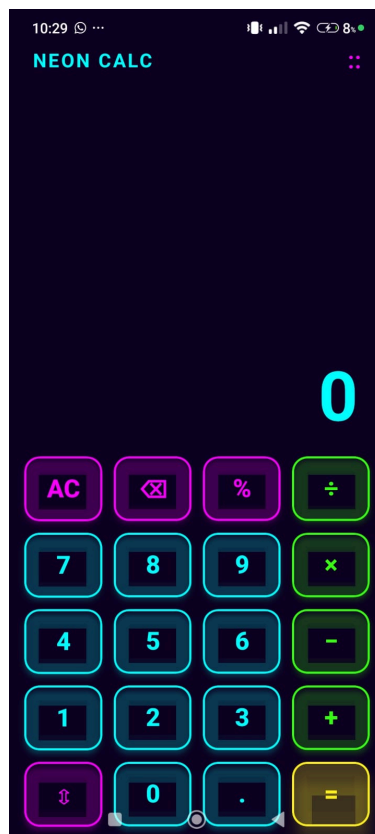
```
2
3 import org.junit.Assert.assertEquals
4 import org.junit.Test
5
6 class CalculadoraTest {
7
8     6 Usages
9     private val calculadora = Calculadora()
10
11     @Test
12     fun verificarSuma() {
13         assertEquals( expected = 8.0, actual = calculadora.sumar( a = 5.0, b = 3.0), delta = 0.001)
14     }
15
16     @Test
17     fun verificarResta() {
18         assertEquals( expected = 2.0, actual = calculadora.restar( a = 5.0, b = 3.0), delta = 0.001)
19     }
20
21     @Test
22     fun verificarMultiplicacion() {
23         assertEquals( expected = 15.0, actual = calculadora.multiplicar( a = 5.0, b = 3.0), delta = 0.001)
24     }
25
26     @Test
27     fun verificarDivision() {
28         assertEquals( expected = 2.0, actual = calculadora.dividir( a = 10.0, b = 5.0), delta = 0.001)
29     }
30
31     @Test
32     fun verificarDivisionPorCero() {
33         assertEquals( expected = 0.0, actual = calculadora.dividir( a = 10.0, b = 0.0), delta = 0.001)
34     }
35 }
```

MyApplication > app > src > test > java > com > kaae > myapplication > CalculadoraTest



3. Prueba de Interfaz

```
1 package com.kaae.myapplication
2
3 import ...
4
5
6
7
8 class PantallaCalculadoraTest {
9
10     14 Usages
11     @get:Rule
12     val composeTestRule = createComposeRule()
13
14     @Test
15     fun verificarSumaEnInterfaz() {
16         composeTestRule.setContent {
17             PantallaCalculadora()
18         }
19
20         // Ingresar números usando testTags
21         composeTestRule.onNodeWithTag( testTag = "input1").performTextInput( text = "10")
22         composeTestRule.onNodeWithTag( testTag = "input2").performTextInput( text = "5")
23
24         // Clic en el botón "Sumar"
25         composeTestRule.onNodeWithText("Sumar").performClick()
26
27         // Verificar resultado (10 + 5 = 15)
28         composeTestRule.onNodeWithTag( testTag = "resultado").assertTextEquals( ...values = "15")
29     }
30
31     @Test
32     fun verificarBotonAC() {
33         composeTestRule.setContent {
34             PantallaCalculadora()
35         }
36     }
37 }
```



3. Análisis de Prueba

A. ¿Qué válida la prueba unitaria?

La prueba unitaria tiene como objetivo principal validar la lógica de negocio en su estado más puro. En este proyecto, se aplicó sobre la clase Calculadora.kt para verificar que los algoritmos matemáticos funcionen con absoluta precisión.

- Validación técnica: Asegura que ante entradas específicas (ej. 10 y 5), la función devuelva exactamente el valor esperado (ej. 15 en suma o 50 en multiplicación).
- Casos críticos: Permite probar situaciones límite, como la división entre cero o el manejo de números decimales y negativos, sin necesidad de interactuar con la pantalla.

B. ¿Qué válida la prueba de interfaz?

La prueba de interfaz, realizada con las herramientas de Jetpack Compose, validó la capa de presentación y la interactividad de la aplicación.

- Flujo del usuario: Verifica que al presionar físicamente (de forma simulada) el botón "7", el número aparezca en el display.
- Integración: Confirma que la interfaz esté correctamente conectada con la lógica; es decir, que al presionar el botón de operación y el botón de igual, el resultado calculado se refleja visualmente en el componente de texto correspondiente.
- Semántica: Asegura que los elementos sean localizables mediante etiquetas de prueba (test Tag), garantizando que la estructura de la UI sea sólida.

C. Diferencias entre ambos tipos de pruebas

Característica	Prueba Unitaria (Unit Test)	Prueba de Interfaz (UI Test)
Entorno	Se ejecuta en la JVM (computadora).	Se ejecuta en un dispositivo o emulador.
Velocidad	Instantánea (ideal para ciclos rápidos).	Más lenta (requiere cargar la app y la UI).
Objetivo	Probar el "Cerebro" (Cálculos/Lógica).	Probar el "Cuerpo" (Botones/Visualización).
Dependencia	No requiere de Android ni de la interfaz.	Depende totalmente del framework de Android.

Mantenimiento	Fácil de actualizar si cambia la lógica.	Requiere ajustes si cambia el diseño visual.
----------------------	--	--

4. Actividades complementarias

1. Verificar una operación de resta.



2. Verificar una operación de multiplicación.



3. Verificar una operación de división.



4. Validar el comportamiento ante números negativos.



5. Preguntas de Reflexión

¿Cuál es la finalidad de las pruebas unitarias en el desarrollo de aplicaciones móviles?

Básicamente, son nuestro "seguro de vida" como desarrolladores. Su finalidad es asegurarnos de que cada pequeña pieza del código (como una suma o una validación) haga exactamente lo que debe hacer por sí sola. En el mundo móvil, donde las apps crecen rápido, estas pruebas nos permiten hacer cambios o añadir funciones sin el miedo constante de haber roto algo que ya funcionaba bien.

¿Qué ventajas ofrece Jetpack Compose para realizar pruebas de interfaz?

Lo mejor de Compose es que "entiende" la interfaz de una manera muy lógica. A diferencia del sistema antiguo donde era un dolor de cabeza encontrar un botón, aquí podemos poner etiquetas claras (testTags) y simular clics o gestos casi como si fuéramos un usuario real. Además, nos permite aislar solo un trocito de la pantalla para probarlo, lo que hace que las pruebas sean mucho más limpias y fáciles de escribir.

¿Qué problemas pueden detectarse mediante pruebas automatizadas?

Uff, de todo. Desde el clásico error matemático (como que un descuento se aplique mal), hasta botones que se quedan "congelados" o textos que se amontonan porque el número es muy largo. También son geniales para detectar esos "errores tontos" que cometemos al final del día por cansancio y que, si no fuera por la prueba automática, acabarían llegando al usuario final.

¿Por qué es importante ejecutar pruebas antes de publicar una aplicación?

Porque no hay nada que dé peor imagen que una app que se cierra sola o que falla en lo más básico el primer día de lanzamiento. Publicar sin probar es como lanzar un coche a la carretera sin revisar los frenos. Probar antes nos da la tranquilidad de que lo que estamos entregando es un trabajo de calidad y que no vamos a tener que estar pidiendo disculpas y lanzando parches de emergencia a las pocas horas.

¿Qué diferencia existe entre probar la lógica de negocio y probar la interfaz gráfica?

Es como la diferencia entre revisar el motor de un coche y revisar el tablero. Probar la lógica es verificar que los engranajes internos funcionen (que el cálculo sea exacto). Probar la interfaz es verificar que, cuando el usuario mueva la palanca, el coche realmente responda. La lógica es el "qué hace" y la interfaz es el "cómo lo ve y lo usa" la persona. Necesitas ambas para que la experiencia sea completa.